

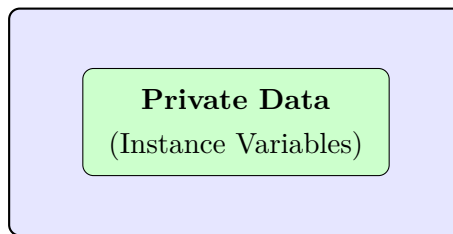
Lecture Review (Extended)

A **class** in C++ can be used to implement an *abstract data type* (ADT). As discussed in lecture, the first step in designing an ADT is to decide upon the *API*, i.e., the set of operations that can be performed on the data. The next step is to decide upon the *representation* of the data. The *instance variables* (also called *attributes*) of the class are used to represent the data.

The representation of the data is hidden from the user of the ADT. The user can only access the data through the operations defined in the API. This is called *data encapsulation* or *information hiding*. The attributes and methods of a class are collectively called the *members* of the class.

Encapsulation: Conceptual View

Public Interface (API)



Encapsulation is achieved by making instance variables **private** and providing controlled access through **public** member functions.

Constructors

A *constructor* is a special member function that is automatically invoked when an object of a class is created. It has the same name as the class and no return type.

Constructors are used to:

- Initialize object attributes
- Ensure the object starts in a valid state

For example, consider a class implementing *rational numbers*.

```
class Rational {
private:
    int num, den;

public:
    Rational() : num{0}, den{1} {}
    Rational(int n) : num{n}, den{1} {}
```

```

    Rational(int n, int d) : num{n}, den{d} {}
    Rational(const Rational& r) : num{r.num}, den{r.den} {}
};

```

Copy Constructors: Shallow vs Deep Copy

When an object contains dynamically allocated memory, a copy constructor must be carefully designed.

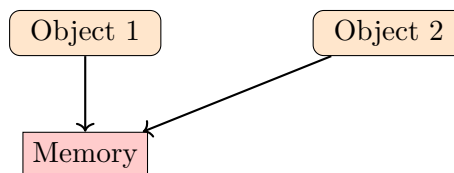
Shallow Copy

A **shallow copy** copies only the memory address, not the data itself.

```

class Shallow {
public:
    int* data;
    Shallow(int v) {
        data = new int(v);
    }
    Shallow(const Shallow& s) { // shallow copy
        data = s.data;
    }
};

```



⚠ Shallow copy is acceptable only when memory is intentionally shared.

Deep Copy

A **deep copy** creates a new independent copy of the data.

```

class Deep {
public:
    int* data;
    Deep(int v) {
        data = new int(v);
    }
    Deep(const Deep& d) { // deep copy
        data = new int(*d.data);
    }
};

```



✓ Deep copy is mandatory when objects **own** their memory.

Operator Overloading

Operator overloading allows objects to behave like built-in data types.

Example: Overloading +

```

class Rational {
private:
    int num, den;

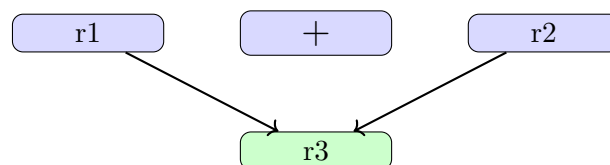
public:
    Rational(int n=0, int d=1) : num{n}, den{d} {}

    Rational operator+(const Rational& r) const {
        return Rational(num + r.num, den + r.den);
    }
};
  
```

Usage:

```

Rational r1(1,2), r2(3,4);
Rational r3 = r1 + r2;
  
```



Encapsulation Example Revisited

```

class Student {
private:
    int erpID;
    string name;
    float cgpa;
  
```

```
public:
    Student(int id, string n, float c) {
        erpID = id;
        name = n;
        cgpa = c;
    }
    void setCGPA(float c) {
        cgpa = c;
    }
    int getErpID() { return erpID; }
    string getName() { return name; }
    float getCGPA() { return cgpa; }
};
```

The public interface exposes only safe operations, ensuring **data integrity** and **controlled access**.

Exercise 1

In this exercise, you will improve the class **Rational** defined in previous section by adding more methods. First, add the following constructors to the class **Rational**.

- A constructor that takes a **string** argument and initializes the numerator and denominator to the values specified in the string. For example, the string "3/4" should initialize the numerator to 3 and the denominator to 4.
- A constructor that takes a floating point number as an argument and initializes the numerator and denominator to the closest rational number to the argument. For example, the floating point number 0.75 should initialize the numerator to 3 and the denominator to 4.

Furthermore, add the following methods to the class **Rational**. Note that some of the following methods are declared as **const**, which means that the method does not modify the object on which it is called.

Method	Description
int get_num() const	Returns the numerator of the rational number.
int get_den() const	Returns the denominator of the rational number.
void set_num(int n)	Sets the numerator of the rational number to n.
void set_den(int d)	Sets the denominator of the rational number to d.
string to_string() const	Converts the rational number to string in the form " num/den ".
void reduce()	Reduces the rational number to its lowest terms. For example, if the rational number is 4/6, then it should be reduced to 2/3.
Rational operator+ (const Rational& r) const	Returns the sum of the current rational number and the rational number r passed as an argument. The result should be in reduced form.
Rational operator- (const Rational& r) const	Returns the difference of the current rational number and the rational number r passed as an argument. The result should be in reduced form.
Rational operator* (const Rational& r) const	Returns the product of the current rational number and the rational number r passed as an argument. The result should be in reduced form.
Rational operator/ (const Rational& r) const	Returns the quotient of the current rational number and the rational number r passed as an argument. The result should be in reduced form.
bool operator== (const Rational& r) const	Returns true if the current rational number is equal to the rational number r passed as an argument, and false otherwise. (Rational(1, 2)== Rational(2, 4) should return true)
bool operator!= (const Rational& r) const	Returns true if the current rational number is not equal to the rational number r passed as an argument, and false otherwise.

Write a client program that tests all the methods of the class **Rational**.

Exercise 2

TicTacToe: Create a **class** **TicTacToe** that will enable you to write a complete program to play the game of tic-tac-toe. The class contains as private data a 3-by-3 two-dimensional array of integers. The constructor should initialize the empty board to all zeros. Allow two human players. Wherever the first player moves, place a 1 in the specified square. Place a 2 wherever the second player moves. Each move must be to an empty square.

The class should have the following public methods.

Method	Description
TicTacToe()	Default constructor. Initialize the board to all zeros.
void clearBoard()	Clear the board to all zeros.
void to_string()const	Return a string representation of the current board.
int gameStatus()const	Returns 1 or 2 if the player 1 or 2 wins respectively, 3 if the game is draw, and 0 if the game is not over yet.
bool move(int player, int row, int col)	Play a move for the player player at the row row and column col . Returns true if the move is successful, and false if the square is not empty.

If you feel ambitious, add a method **void autoMove(int player)** so that the computer makes the moves for **player**.

Write a client that runs a complete game of tic-tac-toe. Allow the two players to alternatively make the moves. After each move, determine whether the game has been won or is a draw. Following is a sample run of the program:

Welcome to Tic-Tac-Toe!

Play 1 move: enter row and column: 0 0

```

  1 | 0 | 0
  -----
  0 | 0 | 0
  -----
  0 | 0 | 0

```

Play 2 move: enter row and column: 1 1

```

  1 | 0 | 0
  -----
  0 | 2 | 0
  -----
  0 | 0 | 0

```

Play 1 move: enter row and column: 0 1

```

  1 | 1 | 0
  -----
  0 | 2 | 0
  -----
  0 | 0 | 0

```

Play 2 move: enter row and column: 1 2

```

  1 | 1 | 0
  -----
  0 | 2 | 2
  -----
  0 | 0 | 0

```

Play 1 move: enter row and column: 0 2

```

  1 | 1 | 1
  -----
  0 | 2 | 2
  -----
  0 | 0 | 0

```

Player 1 wins!

Exercise 3

Create class **IntegerSet** for which each object can hold integers in the range 0 through 100. Represent the set internally as a **vector** of **bool** values. Element **a[i]** is **true** if integer **i** is in the set. Element **a[j]** is **false** if integer **j** is not in the set.

The default constructor initializes a set to the so-called empty set, i.e., a set for which all elements contain **false**.

- Provide member functions for the common set operations. For example, provide a **unionOfSets** member function that creates a third set that is the set-theoretic union of two existing sets (i.e., an element of the result is set to **true** if that element is **true** in either or both of the existing sets, and an element of the result is set to **false** if that element is **false** in each of the existing sets).
- Provide an **intersectionOfSets** member function which creates a third set which is the set-theoretic intersection of two existing sets (i.e., an element of the result is set to **false** if that element is **false** in either or both of the existing sets, and an element of the result is set to **true** if that element is true in each of the existing sets).
- Provide an **insertElement** member function that places a new integer **k** into a set by setting **a[k]** to **true**. Provide a **deleteElement** member function that deletes integer **m** by setting **a[m]** to **false**.
- Provide a **to_string** member function that returns a set as a **string** containing a list of numbers separated by spaces. Include only those elements that are present in the set (i.e., their position in

the vector has a value of **true**). Return empty string "" for an empty set.

- Provide an **isEqualTo** member function that determines whether two sets are equal.
- Provide an additional constructor that receives an array of integers, and uses the array to initialize a set object.

Now write a driver program to test your **IntegerSet** class. Instantiate several **IntegerSet** objects. Test that all your member functions work properly.

Exercise 4

A software company is developing a user management system for an application. Each user has basic profile information as well as login session data.

The system distinguishes between:

- **Permanent user data**, which must be completely independent when a user object is copied.
- **Session-related data**, which may be shared between copied user objects to save memory.

1. Create a class called **User** that contains the following data members:
 - **age** (type **int**)
 - **name** (type **string**)
 - **profileBio** (type **char***), representing dynamically allocated profile information
 - **sessionID** (type **int***), representing shared login session data
2. Provide a parameterized constructor that initializes all data members. Dynamically allocate memory for **profileBio** and **sessionID**.
3. Implement a **copy constructor** such that:
 - A **deep copy** is performed for **profileBio**, ensuring that each **User** object has its own independent copy of the profile information.
 - A **shallow copy** is performed for **sessionID**, allowing multiple **User** objects to share the same session data.
4. Explain briefly why a deep copy is mandatory for **profileBio** but a shallow copy is acceptable for **sessionID**.
5. Write a test application that:
 - Creates a **User** object with name "Ali" and age 22.
 - Creates a second **User** object using the copy constructor.
 - Modifies the profile bio of one object and shows that the other object is not affected.
 - Modifies the session ID and shows that the change is reflected in both objects.

Exercise 5

A university management system stores important dates such as a students date of birth and admission date. For security and encapsulation reasons, the system does not allow direct public access to the individual components of a date. Only a trusted system component is permitted to access the internal date information.

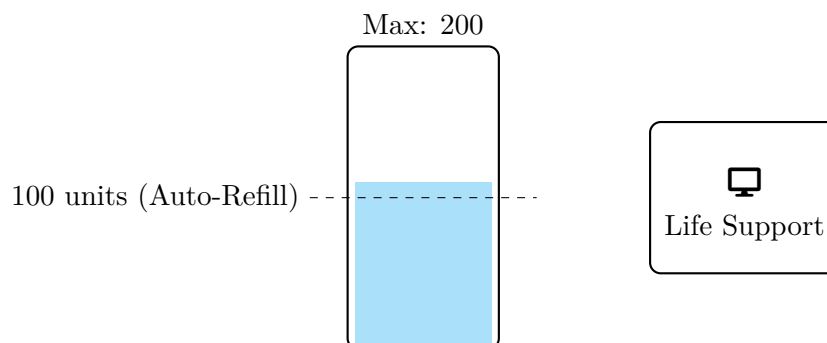
- Create a class called **Date** that contains three private data members: **month** (type **int**), **day** (type **int**), and **year** (type **int**).
- Provide a constructor that initializes the date with given values.
- Provide a member function **displayDate()** that displays the date in the format **mm/dd/yyyy**.
- Create another class called **AgeCalculator** that is responsible for calculating a persons age using the date of birth.
- The **AgeCalculator** class must be able to access the private data members of the **Date** class. Use the **friend** keyword where appropriate to achieve this without violating encapsulation.
- Write a test application named **DateTest** that creates objects of the **Date** and **AgeCalculator** classes and demonstrates their functionality.

Exercise 6

Game Design Scenario: Space Station Oxygen Monitor 🧑‍🚀 ☁

You are designing a sci-fi simulation game called *Orbital Life Support*, where an autonomous system monitors oxygen tanks for astronauts aboard a space station.

Each astronaut has a personal oxygen tank with a maximum capacity of **200 units**. As astronauts perform activities, oxygen is consumed. Whenever the oxygen level drops below **100 units**, the system immediately refills the tank back to full capacity.



Your Task:

1. Create a class named **OxygenTank**.
2. The class should contain:
 - A public integer data member **oxygenLevel**.
 - A method **consume(int units)** that reduces the oxygen level.
 - A method **refill()** that restores oxygen to 200 units.
3. Initially, the tank should be full.
4. If oxygen falls below 100 units, the system must automatically refill it.
5. In **main()**, repeatedly read oxygen consumption values until the user exits.

Exercise 7

Simulation Scenario: Pro Gamer League Player Contract System 🎮 🏆

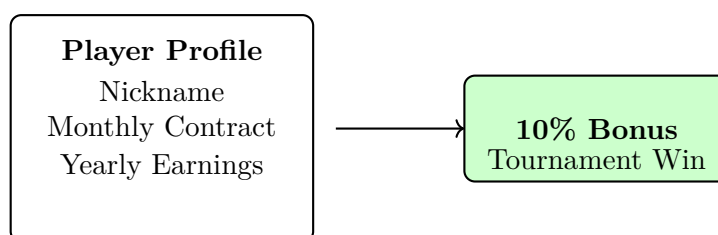
You are developing a management system for an international e-sports league. Each professional player signs a monthly contract and earns rewards based on performance.

Each player has:

- First name
- Last name
- Monthly contract amount

League rules:

- If the monthly contract value is not positive, it is set to **0.0**.
- Yearly earnings are calculated as **12 * monthly contract**.



Your Task:

1. Create a class named **Player**.
2. The class should have private instance variables:
 - `String firstName`
 - `String lastName`
 - `double monthlyContract`
3. Validate that the contract value is non-negative.
4. Implement a method to compute yearly earnings.
5. Write a test program **PlayerTest** that:
 - Creates two players
 - Displays yearly earnings
 - Increases contracts by 10%
 - Displays updated yearly earnings

Exercise 8

Game Scenario: Artifact Vault Museum Collection Manager 🏛️💎

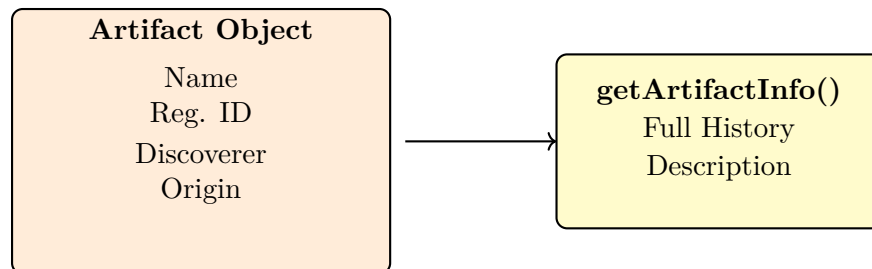
You are creating a digital system for a fantasy museum where rare artifacts are cataloged and displayed for visitors.

Each **Artifact** stored in the vault has:

- Artifact name
- Registration ID

- Discoverer name
- Origin location

Visitors can inspect artifacts individually or view their full historical description.



Your Task:

1. Create a class named **Artifact**.
2. Use private instance variables:
 - `String name`
 - `String registrationID`
 - `String discoverer`
 - `String origin`
3. Use the `this` keyword in constructors and methods.
4. Provide getter methods for all attributes.
5. Implement `getArtifactInfo()` returning full details.
6. Write a test program **ArtifactTest** that:
 - Creates an array of 5 artifacts
 - Displays complete information for each

Exercise 9

You are tasked with designing a simple **Bank Account** system in C++. Each bank account has the following properties:

- An account holder's **name**.
- A unique **account number**.
- The current **balance**.

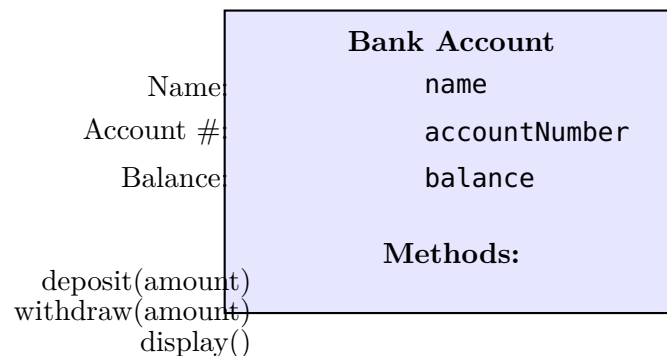
Each bank account should allow the following operations:

- **Deposit** an amount into the account.
- **Withdraw** an amount from the account.
- **Display** account information.

1. Implement a C++ class called **BankAccount**.

2. Use **private instance variables** for `name`, `accountNumber`, and `balance`.
3. Provide **public get and set functions** for `name` and `accountNumber`.
4. Provide public member functions for `deposit`, `withdraw`, and `display`.
5. Use the **this** keyword appropriately to distinguish instance variables from parameters.

Visual Representation of the Bank Account:



Exercise 10

You are developing a simple Football Match Simulation in C++. In this system, each football team is represented as an object. Every team has a team name, the number of goals scored, and the total points earned in the league. The goals and points represent the internal state of the team and should not be directly accessible from outside the class.

A football team can score goals during a match, which increases its goal count. Based on match results, the team can also gain or lose league points. At any time, the team should be able to display its current status, including its name, goals scored, and total points.

Your task is to design a class called `FootballTeam` that models this behavior using object-oriented programming principles in C++.

The class should store the number of goals and league points as private data members, while the team name may be accessed and modified through public member functions. The class must provide a public interface that allows goals to be added, points to be updated, and the current team status to be displayed.

The `FootballTeam` class should support two constructors. The default constructor should initialize a team with zero goals and zero points. A parameterized constructor should allow the team name, initial goals, and initial points to be set at the time of object creation. A destructor must also be implemented, which prints a message when a team object is destroyed, indicating that the team has been removed from the tournament.

To simulate league mechanics, overload the `+` operator so that the points of one team can be added to another team, representing a scenario such as bonus points or merged statistics. Additionally, implement a friend function that compares the goals scored by two teams and determines which team performed better in the match.

